



American University of Beirut
Maroun Semaan Faculty of Engineering & Architecture
Department of Electrical and Computer Engineering

EECE 350 – Computer Networks
Network Programming Project

Python Arena

Instructor: Dr. Sanaa Sharafeddine

Carla Jaffal, 202510287, caj11@mail.aub.edu
Celine Hraibe, 20250232, crh11@mail.aub.edu
Rachel Al Skaff, 202501261, raa209@mail.aub.edu

Workload Distribution
Carla Jaffal, Server Implementation, 33%
Celine Hraibe, Client Implementation, 33%
Rachel Al Skaff, GUI Design and Implementation, 33%

April 26, 2026

1. System and Protocol

System Architecture

- The system follows a client-server architecture. A central server hosts the game session, while multiple clients connect to it by entering the server's IP address and port number.
- Each server manages its own connected users, so only players who connect to the same server can see each other, join the same lobby, send challenges, chat, and participate in matches together.
- The server is responsible for managing the game logic and maintaining the global game state. This includes player connections, unique usernames, active challenges, spectators, movement updates, health values, obstacles, pies, and remaining game time. Since several clients may connect at the same time, the server handles concurrent users using multithreading and ensures that all players receive consistent game updates.
- The client allows the player to enter connection details, choose a username, select his/her snake appearance and control keys, view the lobby, send or accept challenges, watch a game, play the game, use the chat box, and view the final results. During a game, the client sends the player's input, such as movement commands and chat messages, to the server. The server then sends the updated game state and displays it visually on the player's screen.

Protocol Between Communicating Entities

- Communication between the client and server is implemented using TCP sockets. The player enters a server's IP address and port number on the welcome screen, and the client uses these values to open a TCP connection to that specific server. This allows different servers to host separate groups of users and matches.
- All exchanged messages are written in JSON format and sent as newline-separated strings. Each message contains a "type" field that tells the receiver how to process it. For example, the client sends "join" with the username, "challenge" with the other player, "accept" to accept a challenge, "input" for movement commands, "chat" for messages, and "spectate" to request spectator mode.
- The server responds with messages such as "username_ok", "username_taken", "player_list", "challenged", "go_to_setup", "go_to_arena", "waiting_for_opponent", "game_start", "game_state", "chat", "matches_list", "spectate_start", "game_over", and "error". During a match, the server repeatedly broadcasts the current game state, including both snakes, pies, obstacles, health values, remaining time, selected arena, and player names.
- This protocol keeps the client, and server roles separate: the client sends player desires, while the server determines the official result and returns the updated state.

Justification of Design Choices

- A client-server model was used because the game depends on one shared state of the ongoing match. The server is the official source that tracks both snakes' real positions, each player's health, the timer, the obstacles, and the pies. Clients send player inputs, while the server updates and distributes the game state. This prevents each player's screen from showing a different version of the game.

- Using TCP is the appropriate choice because the game messages must arrive in order. A movement command, chat message, challenge request, or game-over update should not be lost or processed out of sequence.
- JSON was chosen because each message has several named fields, such as "type", "username", "target", "direction", "arena", "health", and "time_left". This makes the protocol easy to read, debug, and extend in Python without designing a custom format.
- The server uses multithreading so that each connected client can be handled separately while the main game loop continues running. This allows multiple players to remain in the lobby, receive player-list updates, send challenges, chat, or spectate without blocking one another.
- The protocol can be extended to include arena selection in the game state, allowing Beirut, Byblos, Baalbek, and Sidon arenas to appear without changing the basic socket structure.

2. Feature Implementation Table

The following features were fully implemented:

Feature	Description
Client-server Connection	Clients establish a TCP connection by entering a server's IP address and port number on the welcome screen.
Username Validation	When a client sends a "join" request, the server checks whether the username is non-empty and not already in use. If valid, the user is registered; otherwise, the server rejects it with a username is taken message in the welcome page.
Online Users Tracking (Lobby)	The server maintains a live list of connected users and broadcasts "player_list" updates whenever a user joins or disconnects.
Player Challenging	A "challenge" message creates a pending request, and a "accept" message validates it before transitioning both players to the snake and keys selection page.
Arena Selection	The challenging player selects the arena and the server stores this selection and includes it when initializing the match, ensuring both players load the same environment.
Snake Customization and Controls Setup	The client allows players to choose snake colors and define any control keys. These settings are saved locally and are displayed in the game interface and used during input handling.
Snake Movement Control	Clients send movement commands. The server validates them and applies them during the game loop. This prevents illegal moves (reverse movements) and ensures both players follow the same rules.

Collision Detection	All collisions are computed on the server, and it updates the health dynamically: - Wall collisions reset direction and reduce health (-10) - Collisions with the opponent reduce health (-10) - Obstacles (rocks) reset direction and damage health (-10)
Chat Box	Players can send messages to each other during gameplay. The server relays chat messages to both players, including the sender, so that both game interfaces display the same conversation.
Spectator Mode	Clients who join as spectators receive the same game state updates as players but do not send any movement input.
GUI Rendering using Pygame	The server sends information about snake positions, pies, obstacles, health values, timer, and match status. The client then uses Pygame to visually display this information on the game screen and handle the user's interactions like keyboard inputs and chat typing.
Disconnection Handling	If a player disconnects during a match, the server immediately ends the game and declares the remaining player as the winner.
Return-to-Lobby Flow	If a user wishes to go back to the lobby after playing/watching a match, he/she stays connected to the server, and is transitioned to the lobby window under the same username.

3. Implementation of Functionalities

Client Implementation

The client was implemented in Python and handles the user interface, player inputs, and communication with the server. It includes the connection page, lobby, arena selection, snake setup, gameplay screen, and game-over results. During the match, the client sends movement commands to the server and displays the updated game state, including snakes, pies, rocks, health, timer, chat, and final statistics.

Celine worked on the client code, including the chat box and snake color selection. Rachel implemented the Pygame GUI, integrated it with the client-server logic, and managed setup features such as speed, controls, and arena selection across the system.

Server Implementation

The server was implemented in Python using TCP sockets and threading. It handles multiple clients, checks usernames, manages the lobby, matches players, and broadcasts updates. Carla implemented the server logic, including the main game state, movement processing, collisions, health updates, pie rewards, and rock damage.

Game Logic

The game logic runs mainly on the server to keep both players running smoothly. Pies are generated during the game, while rocks are placed as fixed obstacles. The game ends when a player's health reaches zero or when the one-minute timer ends, with the higher-health player winning.

Networking and Synchronization

The game uses a server-authoritative model. Clients send inputs such as movement, chat, setup choices, and speed selection, while the server processes the game state and sends updates back to both players and spectators. This keeps both players and spectators seeing the same match state at the same time.

Advanced Features

The theme was developed collaboratively by the team as a Lebanese-style game. Rachel extended this idea by associating the pie collectibles with a selected arena: Beirut uses an AUB-themed item, Baalbek has a سفينة بعلبك, Sidon has soap, and Byblos uses a fan. She implemented the arena selection screen in `arena_setup.py`. The selected arena is saved in `selected_arena.txt` and passed through the setup flow. In `game.py`, the selected arena determines the background image using the `ARENA_BACKGROUND` dictionary, which maps Beirut, Baalbek, Sidon, and Byblos to their corresponding image files.

Celine selected the background music, Abu Ali by Ziad Rahbani, to reinforce the Lebanese atmosphere of the game subtly. In the code, music is handled using `pygame.mixer`. The game loads `music.mp3` first, then plays it in a loop with controlled volume.

Celine implemented the chat feature, where in `client.py`, messages are sent through `send_chat()`, which packages the text as JSON and sends it to the server. The server receives the chat message and broadcasts it to the connected players/spectators. In `game.py`, the chat panel displays the received messages and includes an input box for typing during gameplay. She also included a `spectate()` method that sends a "spectate" request to the server. On the server side, spectators are stored separately from active players, then included when broadcasting game-state updates. This allows spectators to receive the same live updates without sending movement inputs or affecting the match.

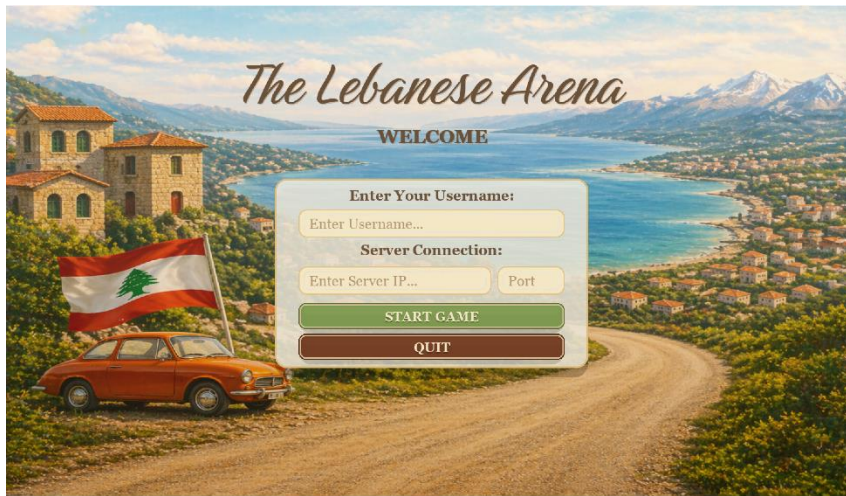
Carla implemented snake color selection in `snake_setup.py`. The available colors are stored in the `SNAKE_COLORS` dictionary, with separate RGB values for the head and body. The selected colors are saved in `player_setup.json`, sent through `setup_ready`, stored by the server as part of `player_styles`, and later used by `game.py` when rendering each snake.

Celine implemented control key customization in `snake_setup.py`. The user-chosen keys are captured through the setup interface and stored in `player_setup.json`. These keys are sent to the server via the `setup_ready` message, saved as part of the player configuration, and then displayed and used by `game.py` to control the snake.

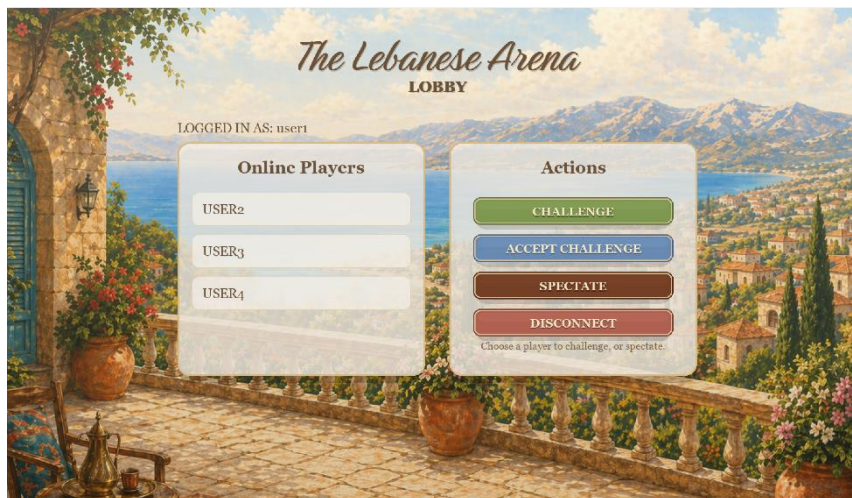
Rachel added the speed feature in the later stages of our testing and designing across the setup, client, and server. The selected speed is included in the `setup_ready()` message in `client.py` as `"speed": speed`. The server receives this setup data and applies the selected speed to the game loop tick rate, so both players experience the same pace instead of having separate local speeds.

The game tracks match results beyond just the winner. Carla made sure that the server includes final health and statistics in the `"game_over"` message, while `game.py` displays the final result screen. This includes information such as the winner, remaining health, pies collected, and obstacles hit, informing the user of some match statistics.

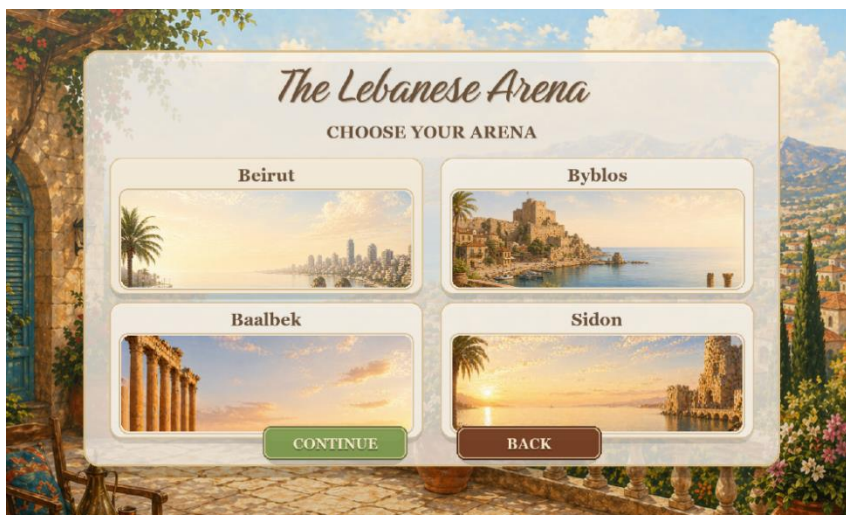
Appendix 1



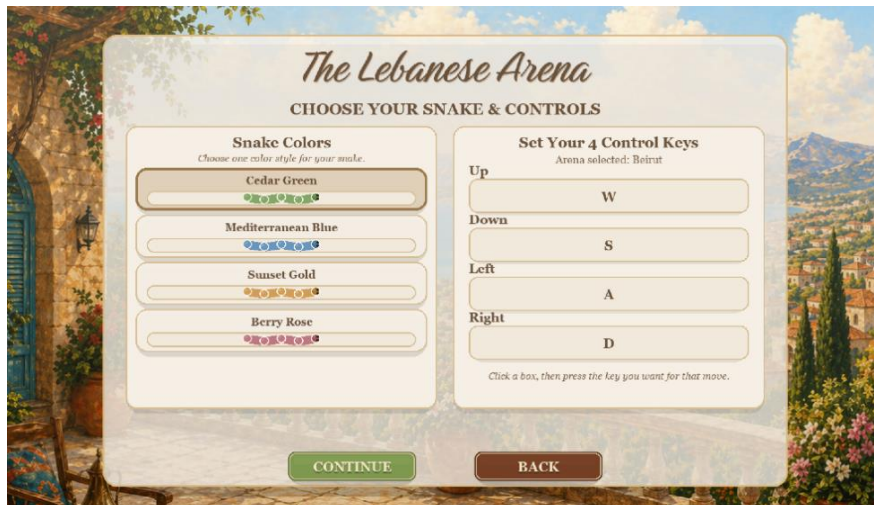
Welcome page: users enter their username and establish a connection to the server by specifying the IP address and port number.



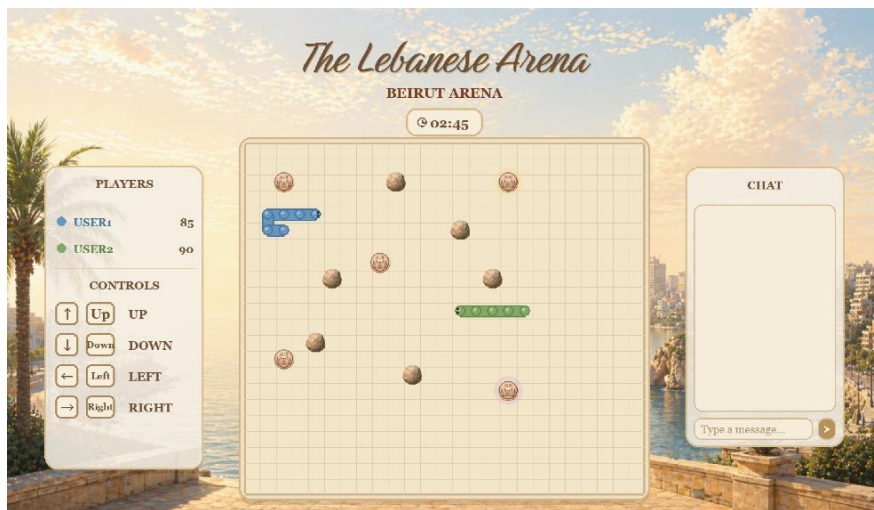
Lobby: connected users can view online players and initiate or receive game challenges.



Arena selection page: only the challenging player chooses the game environment.



Snake customization:
players select snake
colors and define key
bindings for movement.



Gameplay screen:
includes the grid, player-
controlled snakes,
obstacles, scores, and
remaining time.

Appendix 2

Task Division

Task	Description	Responsible
Client Core Logic	TCP connection, JSON messaging, client state, player list, challenges, and game-state reception.	Celine
Server Implementation	Server socket setup, multithreading, shared game state, and broadcasting updates	Carla
Matchmaking	Challenge/accept logic and starting two-player matches	Carla + Celine
Pygame Application Flow	Welcome, lobby, arena setup, snake setup, and gameplay.	Rachel
GUI–Networking Integration	Connecting buttons, screens, setup choices, and gameplay actions to client/server communication.	Rachel
Speed Feature	Implementing speed selection across the client and server.	Rachel
Return-to-Lobby Flow	Maintaining connection and username when returning after game over.	Rachel
Snake Color Selection	Selected snake colors are passed to the game.	Carla
Control Keys Customization	User-selected keyboard controls are passed and used in the game.	Celine + Rachel
Chat Feature	Chat input, sending messages, receiving messages, and displaying them in the game.	Celine + Rachel
Background Music	Integrating <i>Abu Ali</i> by Ziad Rahbani into the game.	Celine
Spectator Mode	Allowing “fans” to watch ongoing matches.	Celine + Carla
Game Logic	Snake movement, collisions, pie collection, health changes.	Carla
Timer and Winner Decision	One-minute countdown and deciding the winner based on remaining health.	Celine + Carla
Final Statistics	Tracking and displaying health, pies collected, and obstacles hit in the game over panel.	Carla
Testing and Debugging	Multiplayer tests, connection tests, UI adjustments, and report writing.	All members